Phase 1 静态 FJSP 数学模型

FJSP-AGV-RL Research Platform

2026 年 6 月 6 日

1 范围

本文档描述仓库当前 Phase 1 实现的静态柔性作业车间调度（Flexible Job Shop Scheduling Problem, FJSP）模型。AGV 运输、路径规划、动态到达、强化学习和图神经网络均不属于 Phase 1 范围。

2 集合与索引

符号	含义
J	工件集合，索引为 j
O_j	工件 j 的有序工序集合，索引为 o
M	机器集合，索引为 m
A_{jo}	工序 (j,o) 的可选机器集合

同一工件内部必须满足工艺顺序：若 $o+1 \in O_j$，则工序 (j,o) 必须先于 $(j,o+1)$ 完成。

3 参数

符号	含义
p_{jom}	工序 (j,o) 在机器 m 上的加工时间
H	保守调度上界

实现中使用如下保守 horizon：

$$H = \sum_{j \in J} \sum_{o \in O_j} \max_{m \in A_{jo}} p_{jom}.$$

该上界不一定紧，但足以支撑当前小规模 CP-SAT baseline。

符号	含义
$x_{jom} \in \{0,1\}$	若工序 (j,o) 分配给机器 m，则为 1
$S_{jo} \geq 0$	工序 (j,o) 的开始时间
$C_{jo} \geq 0$	工序 (j,o) 的完成时间
$C_{\max} \geq 0$	makespan
$y_{abm} \in \{0,1\}$	同一机器上两个工序 a,b 的先后顺序变量

4 决策变量

当前 OR-Tools 实现使用 optional interval variables 和 `NoOverlap`，并不显式建立所有 y 变量；这里给出 y 是为了说明经典数学规划形式。

5 约束

5.1 机器选择约束

每个工序必须且只能选择一台可行机器：

$$\sum_{m \in A_{jo}} x_{jom} = 1, \qquad \forall j \in J,\ o \in O_j.$$

不可行机器不得被选择：

$$x_{jom} = 0, \qquad \forall m \notin A_{jo}.$$

5.2 加工时间约束

若工序选择机器 m，其完成时间由开始时间和加工时间决定：

$$C_{jo} = S_{jo} + \sum_{m \in A_{jo}} p_{jom} x_{jom}.$$

5.3 工序顺序约束

同一工件内部的后续工序不得早于前序工序完成：

$$S_{j,o+1} \geq C_{jo}, \qquad \forall j \in J,\ o, o+1 \in O_j.$$

5.4 机器容量约束

任意机器在同一时间最多加工一个工序。对可能使用同一机器 m 的两个不同工序 $a = (j,o)$ 和 $b = (j',o')$，可用 Big-M 形式表示为：

$$S_b \geq C_a - H(1 - y_{abm}) - H(2 - x_{am} - x_{bm}),$$

$$S_a \geq C_b - H y_{abm} - H(2 - x_{am} - x_{bm}).$$

在代码中，这一约束由 CP-SAT 的 optional interval 和 `NoOverlap` 表达；启发式输出则由独立 feasibility checker 验证。

5.5 Makespan 定义

$$C_{\max} \geq C_{jo}, \qquad \forall j \in J,\ o \in O_j.$$

目标函数为：

$$\min C_{\max}.$$

6 Feasibility Checker 对应关系

代码中的 `src/scheduling/feasibility.py` 对应检查以下内容：

- 所有工序必须被调度且仅调度一次；
- 每个工序的 `job_id`、`op_id`、`machine_id` 必须合法；
- 加工时间必须与实例数据一致；
- 同一工件内部满足 precedence；
- 同一机器上不存在重叠加工；
- 机器必须属于该工序的可选机器集合；
- makespan 必须与记录中的最大完成时间一致。

因此，任何求解器或启发式算法返回调度前，都必须经过 feasibility checker。这也是 AGENTS.md 中 “正确性优先” 的核心要求。

7 已跟踪的 FJSPLib 子集

仓库跟踪了一个小型真实静态 FJSP 子集，文件来自 ScheduleOpt 官方 FJSPLib 仓库，并保留 CC-BY-SA-4.0 归属说明：`https://scheduleopt.github.io/benchmarks/fjsplib`。当前子集覆盖六个来源系列，共 22 个上游文件。

上表中的 CP-SAT 值均通过独立 feasibility checker。若 CP-SAT 状态为 `FEASIBLE` 而非 `OPTIMAL`，该值只是限时可行参考调度，不是本地已证明最优解。当前 `mk02`、`mk03`、`mk04`、`mk05`、`mk06`、`mk07`、`mk08`、`mk09`、`mt10c1`、`01a` 和 `kacem4` 都属于这种情况。

官方 benchmark 表格将 `fattahi6` 标为 3×2，但官方 old-format 文件头为 `3 3 1.6`。本仓库按可加载文件内容记录为 3×3，并保留上游 BKS 320。

8 实验摘要

8.1 CP-SAT benchmark 矩阵

实验 `exp_009` 对 22 个 tracked FJSPLib 实例分别使用单 worker 和 auto worker，在 3 秒时限下各重复 2 次。最新结果为：

实例	工件数	机器数	FJSPLib BKS	本地 CP-SAT 10s
fattahi1	2	2	66	66
fattahi2	2	2	107	107
fattahi3	3	2	221	221
fattahi4	3	2	355	355
fattahi5	3	2	119	119
fattahi6	3	3	320	320
mt10c1	10	11	927	927
01a	10	5	2505	2628
kacem1	4	5	11	11
kacem2	10	7	11	11
kacem3	10	10	7	7
kacem4	15	10	11	15
ft06	6	6	55	55
mk01	10	6	40	40
mk02	10	6	26	28
mk03	15	8	204	220
mk04	15	8	60	62
mk05	15	4	172	180
mk06	10	15	57	74
mk07	20	5	139	159
mk08	20	10	523	523
mk09	20	10	307	340

- 88/88 个 CP-SAT 调度全部通过 feasibility checker；
- 小实例多数能证明 OPTIMAL；
- mk05、mk06、mk07、mk08、mk09、kacem4 等较难实例仍主要表现为限时 FEASIBLE；
- 自动 worker 在本轮观察中通常给出更好的 incumbent 或 bound，但这不是稳定性能排名。

8.2 启发式 benchmark 矩阵

实验 exp_010 在每个 tracked 实例上运行确定性 baseline 与严格 dispatching rule，并对 Random 与 DispatchRandom 使用固定 seed 42、123、999。最新结果为：

- 308/308 个启发式调度全部通过 feasibility checker；
- 没有任何单一 dispatching rule 在所有实例上占优；
- 当前 22 实例上，DispatchFIFO 的平均 best gap 最低；

- `kacem4` 的启发式最好值为 12，仍高于 BKS 11，是后续优化的有用压力样本。

算法族	平均 best gap	最大 best gap
DispatchFIFO	0.135868	0.331176
DispatchMWKR	0.138297	0.350593
DispatchMOPNR	0.180611	0.466667
EarliestFinishTime	0.412865	1.384615
DispatchEFT	0.416947	1.384615

8.3 元数据审计

实验 `exp_012` 会递归发现 tracked FJSPLib 文件，将其与 `instances/fjsplib_bks.csv` 逐项匹配，并验证每个文件仍能被 loader 加载。当前 22/22 个 tracked 文件全部通过文件、BKS 行和 loader 检查。

9 维护结论

当前项目仍然适合继续小步扩展，不需要推倒重构。推荐继续沿着以下顺序推进：

1. 小批量扩展 FJSPLib 静态实例；
2. 每批都运行 metadata audit、heuristic matrix、CP-SAT matrix 和 pytest；
3. 保持 feasibility checker 独立且不可绕过；
4. 在 Phase 1 未稳定前，不引入 AGV、RL 或 GNN。